

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Магистерская программа «Искусственный интеллект»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Исследовательская выпускная квалификационная работа на тему:
Обучение рассуждению без верификаторов для генерации юмора

Выполнил студент:

2 курса магистерской программы

ФИО студента

Принял руководитель ВКР:

ФИО научного руководителя

Должность, ученая степень (при наличии)

Факультет компьютерных наук НИУ ВШЭ

Москва 2026

Оглавление

Аннотация	5
Ключевые слова	6
Введение	7
1 Обзор литературы	10
1.1 Вычислительный юмор как исследовательская проблема	10
1.2 Генерация юмора большими языковыми моделями	12
1.3 Проблема оценки юмора	13
1.4 Рассуждение и творческий поиск в генерации юмора	14
1.5 Ограничения стандартного постобучения	15
1.6 Обучение без внешнего верификатора	16
1.7 Позиционирование данной работы	17
2 Методология	19
2.1 Общая схема метода и программного конвейера	19
2.2 Построение набора данных	20
2.2.1 Удаление дублей и почти совпадающих шуток	20
2.3 Плотный индекс векторных представлений	21
2.4 Извлечение ключевых слов как поиск слабых запросов	22
2.5 Построение эталонных ответов через группировку ключевых слов и поиск	23
2.5.1 Группировка ключевых слов и формирование запросов . .	23
2.5.2 Поиск эталонных ответов с помощью индекса векторных представлений	24
2.5.3 Дедупликация эталонных запросов и разбиение выборки .	25
2.6 Генерация кандидатов	26

2.7	Оценка кандидатов	26
2.7.1	Попарное сравнение	26
2.7.2	Агрегация таблицы результатов	27
2.8	Теоретическая рамка обучения MRVF	28
2.8.1	Обозначения	28
2.8.2	Обучение с верифицируемыми наградами	29
2.8.3	VeriFree	30
2.8.4	Многореференсное обобщение VeriFree	32
2.8.5	Логарифмическая масса как практическая цель	35
2.8.6	Неполнота наблюдаемых эталонных ответов	36
2.8.7	Использование подмножества эталонных ответов	37
2.8.8	Цикл обучения	38
2.8.9	Замечание о базовых уровнях и остановке градиента	40
3	Эксперименты	43
3.1	Цель экспериментальной части	43
3.2	Используемые данные	43
3.3	Обучаемые модели	44
3.4	Сравниваемые варианты моделей	45
3.5	Протокол генерации кандидатов	46
3.6	Протокол автоматической оценки	47
3.7	Сохранение артефактов и воспроизводимость	48
3.8	Статус текущего экспериментального прогона	48
4	Результаты	50
4.1	Текущие результаты	50
4.2	Планируемая процедура обучения	50
4.3	Планируемые абляции	52
	Заключение	53
	Список литературы	55
A	Реализованные шаблоны запросов	59
A.1	Запрос для генерации эталонных ответов	59
A.2	Инструкции для запросов для векторного поиска	59

А.3	Оценочный запрос	60
В	Дополнительные замечания о программной архитектуре	61

Аннотация

Ключевые слова

Введение

Генерация юмора является удобным способом изучения ограничений современных методов постобучения языковых моделей. В таких областях, как математика или программирование, сгенерированный ответ часто можно проверить внешней процедурой: решение либо совпадает с правильным ответом, либо программа проходит тесты или не проходит их. Юмор устроен иначе. Шутка может быть грамматически правильной, связной и соответствующей теме, но при этом не производить нужного эффекта. При этом проблема не сводится только к субъективности юмора. Один и тот же запрос может допускать множество разных удачных шуток, тогда как наблюдаемый набор данных обычно содержит лишь небольшую их часть. Поэтому юмор является не только сложной задачей генерации, но и полезным примером для изучения рассуждения и обучения с подкреплением в неverified доменах [14, 11].

Эта сложность стала особенно важной после появления больших языковых моделей. Ранние системы вычислительного юмора были узкими, но интерпретируемыми: они опирались на шаблоны, лексические ресурсы или конкретные механизмы каламбуров [15, 1]. Современные языковые модели обладают гораздо более широким покрытием, однако недавние работы показывают, что беглость текста не равна сильной юмористической компетентности. Крупные оценки по-прежнему помещают модельные системы ниже сильных человеческих результатов, а исследования каламбуров и понимания шуток показывают, что модели часто надежнее воспроизводят поверхностную форму юмора, чем его внутренний механизм [24, 21, 4, 13]. Иными словами, юмор выявляет знакомую слабость современных моделей: они хорошо продолжают вероятный текст, но хуже выполняют открытый поиск по множеству правдоподобных и неожиданных продолжений.

Один из ответов на эту слабость состоит в явном моделировании процесса рассуждения. Современные системы генерации юмора все чаще используют поэтапное построение подсказок, творческий поиск или декомпозицию на основе

теории, а не одношаговое завершение текста. Эти подходы мотивированы простым наблюдением: хорошие шутки часто возникают не через выбор первого вероятного продолжения, а через исследование ассоциаций, переосмыслений, контрастов и форм панчлайна [2, 25, 17, 10, 23]. Результаты выглядят многообещающе, но имеют ограничение. В большинстве таких систем структура рассуждения задается вручную: исследователь определяет число этапов, тип промежуточных представлений и порядок действий модели.

Параллельное направление в постобучении предлагает другую возможность. В верифицируемых доменах обучение с подкреплением недавно использовалось для извлечения развернутого поведения рассуждения напрямую, а не только через ручное построение подсказок или фиксированные процессы [12, 5]. Методы без внешнего верификатора, такие как VeriFree и RLPR, дополнительно показывают, что часть этой логики может быть перенесена за пределы математики и программирования, если заменить внешнего верификатора внутренними вероятностными целями [26, 22]. Однако для юмора такие формулировки сразу сталкиваются с несоответствием самой природе задачи. Они построены вокруг предположения об одной эталонной ссылке, тогда как юмор естественно является задачей с несколькими эталонными ответами: для запроса вроде “write a joke about a frog” существует много приемлемых ответов, и любой набор данных фиксирует только некоторые из них.

Данная работа мотивирована именно этим несоответствием. Ее центральное утверждение состоит в том, что генерацию юмора следует формулировать не как точную имитацию наблюдаемой шутки, а как распределение вероятностной массы по множеству приемлемых шуток при заданном запросе. Этот сдвиг важен и теоретически, и практически. Теоретически он меняет способ определения обучения без внешнего верификатора с подкреплением в творческом домене. Практически он меняет требования к данным: вместо изолированных пар запрос–ответ обучение требует наборов эталонных ответов, связанных с одним запросом и содержащих несколько правдоподобных шуток.

Работа поэтому состоит из двух связанных частей. Первая часть является методологической и теоретической: в ней формулируется расширение обучения с подкреплением без внешнего верификатора на случай нескольких эталонных ответов для генерации юмора и анализируются последствия неполноты наблюдаемых эталонных наборов. Вторая часть является реализационной: она строит

процесс данных, необходимый для практического применения такой цели. В текущем репозитории этот процесс уже включает сбор корпуса, генерацию плотных векторных представлений, извлечение ключевых слов, построение запросы, поиск эталонных ответов, генерацию кандидатов и автоматическую оценку. Отсутствующим этапом остается сам цикл обучения. Соответственно, данную работу следует читать как предварительное исследование в точном смысле этого термина: она задает формальную цель и программную инфраструктуру, необходимую для последующего обучающего эксперимента.

Источником данных является объединенный корпус примерно из 664k шуток, собранный из коллекций Short Jokes и rJokes. Полный корпус используется как пул поиска, тогда как его ключевых слов достаточно для построения обучающего набора данных с несколькими эталонными ответами. Целевая экспериментальная постановка является узким и контролируемым: модели Qwen3-1.7B и Qwen3-4B в режимах instruct и thinking будут сравниваться с вариантом thinking, обученным с помощью MRVF; обучение планируется на NVIDIA H200 GPU с использованием модифицированного `trl` и `vLLM` для генерации траекторий и вывода базовых моделей.

Глава 1

Обзор литературы

1.1 Вычислительный юмор как исследовательская проблема

Юмор является одной из наиболее трудных задач для обработки естественного языка, поскольку успешная шутка определяется не только грамматической правильностью или тематической релевантностью. Она обычно требует нарушения ожиданий, использования общего знания, выбора подходящей формы, учета аудитории и точного соотношения между подготовкой и неожиданным завершением. Поэтому генерация юмора занимает промежуточное положение между задачами языкового моделирования, творческого письма и прагматического анализа коммуникации.

Ранние исследования вычислительного юмора уже подчёркивали, что юмор плохо сводится к одному формальному правилу. Ritchie [15] описывает вычислительный юмор как область, в которой частные механизмы могут быть формализованы, но общая способность понимать и порождать юмор требует знания о языке, мире и намерениях участников коммуникации. Обзор Amin и Burghardt [1] показывает, что классические системы генерации юмора часто строились вокруг узких механизмов: шаблонов, лексических ресурсов, фонетических замен, каламбуров или заранее заданных форм шутки. Такие системы были интерпретируемыми и позволяли контролировать юмористический механизм, но плохо переносились на открытые темы и разнообразные запросы.

Современные обзоры приходят к сходному выводу уже в контексте больших языковых моделей. Loakman, Thorne и Lin [14] отмечают, что генерация и объяснение юмора остаются недостаточно изученными по сравнению с более стандартными задачами обработки естественного языка, особенно если выхо-

доть за пределы каламбуров. Lemmens и De Marez [11] также подчёркивают, что несмотря на рост возможностей нейросетевых моделей, юмор остаётся областью с неоднозначными результатами: модели способны воспроизводить внешние признаки юмористического текста, но это не означает устойчивого понимания юмористических механизмов.

Каламбуры являются удобным примером этой проблемы. С одной стороны, они дают относительно чёткий объект анализа: в них можно изучать неоднозначность, фонетическое сходство и переключение смысла. Так, Kao, Levy и Goodman [9] формализуют юмор в каламбурах через сочетание неоднозначности и отличительности, показывая, что эти факторы связаны с человеческими оценками смешного. С другой стороны, успех на каламбурах не переносится автоматически на более широкие формы юмора: короткие нарративные шутки, тематические шутки, ситуационный юмор или социально обусловленные ответы. Современные исследования больших языковых моделей подтверждают это ограничение. Ху и др. [21] показывают, что модели часто демонстрируют «ленивую» генерацию каламбуров: вместо компактного совмещения двух значений они явно называют оба связанных понятия или воспроизводят поверхностный шаблон игры слов. Cossieri и др. [4] также показывают, что даже сильные языковые модели испытывают трудности с обобщением в задачах понимания и генерации юмора, когда требуется не просто распознать знакомую форму, а применить юмористический механизм в новой ситуации.

Для данной работы из этого следует два вывода. Во-первых, генерацию юмора нельзя рассматривать как обычную задачу продолжения текста: беглость и тематическая релевантность являются необходимыми, но недостаточными условиями. Во-вторых, юмор является задачей с множеством допустимых ответов. Для одного и того же запроса может существовать большое число хороших шуток, основанных на разных ассоциациях и разных механизмах неожиданности. Это делает постановку с единственным эталонным ответом методологически слабой для обучения и оценки юмористической генерации.

1.2 Генерация юмора большими языковыми моделями

Появление больших языковых моделей изменило характер исследований генерации юмора. В отличие от шаблонных систем, такие модели способны отвечать на широкий спектр запросов и порождать тексты, которые по форме похожи на человеческие шутки. Однако литература показывает, что это расширение охвата не устраняет центральные трудности области.

Крупные корпуса юмора, такие как rJokes, важны как источники разнообразных реальных шуток [19]. Они позволяют изучать тематическое разнообразие, повторяющиеся формы и статистические свойства юмористических текстов. Однако такие корпуса обычно не строятся как наборы данных для многореференсной генерации. В них нет явного соответствия между одним запросом и множеством допустимых ответов. Поэтому прямое дообучение модели на корпусе шуток может научить её стилю и распространённым формам, но не обязательно научит выбирать неожиданный и уместный юмористический ход для нового запроса.

Ограничения современных моделей хорошо видны в задачах, где существует сильная человеческая оценка. Zhang и др. [24] представляют крупный ресурс для генерации подписей к карикатурам, содержащий более 250 миллионов человеческих оценок и более 2.2 миллиона подписей. Эта работа особенно важна, поскольку рассматривает юмор как творческую задачу предпочтений, а не как бинарную классификацию. Авторы показывают, что даже сильные модели и методы дообучения по предпочтениям остаются ограниченными на творческой задаче: лучшие системы всё ещё уступают сильным человеческим участникам.

Похожие выводы появляются и в более специализированных исследованиях. Loakman, Thorne и Lin [13] анализируют понимание юмора от традиционных каламбуров до тематических шуток и показывают, что современные модели не одинаково хорошо справляются с разными типами юмора. Sakabe и др. [16] рассматривают японскую импровизационную форму Oogiri и оценивают ответы по нескольким измерениям: новизне, ясности, релевантности, интеллектуальности, эмпатии и общей смешности. Их результаты показывают, что языковые модели могут достигать уровня между низкими и средними человеческими результата-

ми, но заметно отличаются от людей по тому, какие качества ответа они считают важными.

Эти работы показывают, что прогресс больших языковых моделей не делает задачу генерации юмора решённой. Скорее, он меняет исследовательский вопрос. Если раньше основная трудность состояла в том, чтобы вообще получить грамматически и структурно похожий на шутку текст, то теперь важно отличать поверхностную имитацию от устойчивого творческого обобщения. Для данной работы это особенно существенно: оптимизируемая модель не должна просто запоминать распространённые шутки или воспроизводить корпусные шаблоны, а должна использовать эталонные шутки как слабый сигнал для обучения распределения творческих траекторий.

1.3 Проблема оценки юмора

Оценка юмора является отдельной методологической трудностью. В отличие от задач с объективно проверяемым ответом, смешность зависит от аудитории, контекста, культурных ожиданий, формы подачи и конкретного измерения качества. Поэтому простая формулировка «ответ смешной или нет» часто оказывается слишком грубой.

Корпус rJokes уже демонстрирует эту проблему: реальные шутки разнообразны по теме, форме и качеству, а их восприятие зависит от сообщества, в котором они были опубликованы [19]. Более поздние работы делают этот вывод сильнее. Sakabe и др. [16] показывают, что оценка юмора должна учитывать несколько измерений, а не только общую смешность. Их анализ также показывает расхождение между человеческими и модельными критериями оценки: люди и языковые модели могут придавать разный вес новизне, эмпатии и релевантности.

Winters и Stockt [20] предлагают другой важный взгляд, сравнивая генерацию юмора в условиях, близких к импровизационной комедии. Такая постановка подчёркивает, что юмор не является только свойством изолированного текста. На оценку влияет ситуация, скорость ответа и ожидания аудитории. Следовательно, результаты статической текстовой оценки не следует напрямую отождествлять с качеством юмора в живом взаимодействии.

В то же время полностью отказаться от автоматической оценки в вычис-

лительных экспериментах трудно. Человеческая оценка дорога, шумна и плохо масштабируется. Поэтому современные работы используют языковые модели как вспомогательных оценщиков или строят более структурированные рубрики. Hashemi и др. [8] предлагают многомерный и калиброванный подход к автоматической оценке текстов, показывая, что оценивание с помощью языковых моделей требует явной структуры и контроля. Gunjal и др. [7] рассматривают рубрики как источник награды для обучения за пределами строго проверяемых доменов. Эти подходы важны для данной работы, поскольку юмор также относится к областям, где внешняя награда является неполной и шумной.

Тем не менее модель-оценщик не является прозрачной заменой человеческого восприятия. В данной работе автоматическая попарная оценка используется как воспроизводимый инструмент промежуточного сравнения моделей, а не как окончательное измерение человеческой смешности. Это ограничение принципиально: даже если модель выигрывает в попарной оценке, такой результат следует интерпретировать как улучшение относительно выбранного протокола оценки, а не как доказательство универсального юмористического качества.

1.4 Рассуждение и творческий поиск в генерации юмора

Один из важных сдвигов последних лет состоит в переходе от мгновенной генерации ответа к явному моделированию промежуточного рассуждения. В широком контексте обработки естественного языка Wei и др. [18] показывают, что подсказки с цепочкой рассуждения могут существенно менять поведение больших языковых моделей на задачах, требующих многошагового вывода. Для юмора этот результат нельзя переносить напрямую: юмористическое мышление не всегда является линейным доказательством. Однако сама идея промежуточного поиска оказывается релевантной.

Chen, Shi и Si [2] рассматривают генерацию юмора как процесс, который можно направлять пошаговыми инструкциями, основанными на принципах комедийного письма. Такой подход показывает, что явная организация творческого процесса может улучшать ответы по сравнению с простой генерацией по запросу. Tikhonov и Shtykovskiy [17] развивают похожую идею для коротких

шуток, реконструируя процесс их создания как многошаговый механизм. В их постановке модель не просто выдаёт финальную шутку, а проходит через промежуточные этапы, связанные с поиском предпосылки, неожиданного поворота и окончательной формулировки.

Другие работы подчёркивают, что творческие задачи требуют не только последовательного рассуждения, но и ассоциативных скачков. Zhong и др. [25] предлагают парадигму Leap-of-Thought для генерации юмора в формате Oogiri. В отличие от стандартной цепочки рассуждения, такая схема акцентирует внимание на неожиданных ассоциациях между отдалёнными понятиями. Kim и Chilton [10] описывают генерацию юмора как сочетание когнитивных, социальных и творческих навыков, а Zhang и др. [23] предлагают многоэтапную теоретически направляемую схему для интерпретируемой мультимодальной генерации юмора.

Эти исследования важны для данной работы по двум причинам. Во-первых, они подтверждают, что промежуточные представления и творческий поиск могут быть полезны для генерации юмора. Во-вторых, большинство таких подходов задаёт структуру рассуждения вручную: исследователь заранее определяет этапы, их порядок и типы промежуточных объектов. В данной работе используется другая перспектива. Траектория рассуждения рассматривается как случайная переменная, которую модель может порождать сама. Обучение должно не только повысить вероятность конкретных эталонных шуток, но и изменить распределение траекторий так, чтобы они чаще приводили к правдоподобным юмористическим ответам.

1.5 Ограничения стандартного постобучения

Наиболее прямой способ улучшить генерацию юмора состоит в дообучении модели с учителем на корпусе шуток. Однако такая постановка плохо соответствует структуре задачи. Для одного запроса существует много допустимых шуток, и наблюдаемая в корпусе шутка является только одним из возможных вариантов. Если модель обучается воспроизводить единственный ответ, она может улучшать стилистическую похожесть на корпус, но не обязательно улучшать способность к творческому обобщению.

Общая литература по постобучению усиливает это опасение. Chu и др. [3]

противопоставляют дообучение с учителем и обучение с подкреплением, показывая, что первое может сильнее запоминать обучающие шаблоны, тогда как второе при удачной постановке лучше переносится на новые варианты задач. Для юмора это различие особенно важно: запоминание частотных форм шуток не равно способности построить новый неожиданный панчлайн.

Оптимизация по предпочтениям и обучение с подкреплением по модели награды также имеют ограничения. Gao, Schulman, Hilton и др. [6] показывают, что чрезмерная оптимизация несовершенной модели награды может ухудшать истинное качество. В творческих задачах этот риск особенно велик, потому что модель награды или модель-оценщик неизбежно отражает неполные и шумные критерии. Работа Zhang и др. [24] показывает ограничения RLHF- и DPO-подобных методов для юмористических подписей к карикатурам, а результаты Sakabe и др. [16] и Winters и Stockt [20] подчёркивают расхождение между модельными и человеческими оценками юмора.

Следовательно, стандартные методы постобучения не решают задачу полностью. Дообучение с учителем может усиливать имитацию корпуса, а обучение по внешней модели награды может оптимизировать неполный или смещённый критерий. Для данной работы это мотивирует поиск промежуточной постановки: использовать корпус эталонных шуток как слабый сигнал качества, но не обучаться на нём как на единственно правильных ответах и не вводить отдельную модель награды.

1.6 Обучение без внешнего верификатора

Недавние успехи обучения с подкреплением для рассуждающих языковых моделей во многом связаны с доменами, где доступна объективная проверка ответа. В математике и программировании можно задать верификатор, который проверяет правильность результата. Lightman и др. [12] показывают пользу процессной разметки для математических задач, а DeepSeek-AI [5] демонстрируют, что обучение с подкреплением может вызывать длинные траектории рассуждения даже без заранее написанных человеком цепочек рассуждения. Однако такие результаты опираются на возможность проверить ответ, что плохо переносится на творческие задачи.

Подходы без внешнего верификатора пытаются заменить явную проверку

внутренними вероятностями самой модели. Zhou и др. [26] предлагают VeriFree: вместо того чтобы генерировать ответ и проверять его внешним модулем, метод максимизирует вероятность эталонного ответа при условии запроса и сгенерированной траектории рассуждения. В такой постановке вероятность эталонного ответа одновременно играет роль сигнала награды для траектории рассуждения и источника прямого градиента для увеличения правдоподобия эталонного ответа.

Близкую идею развивают Yu и др. [22] в методе RLPR. Они используют внутренние вероятностные оценки модели как сигнал награды и подчёркивают необходимость стабилизации такого сигнала, поскольку вероятности длинных ответов имеют высокую дисперсию и могут становиться численно неудобными. Этот вывод особенно важен для генерации юмора. В задачах с коротким ответом вероятность эталонной последовательности ещё может быть практическим сигналом. Для полной шутки, состоящей из десятков токенов, произведение токеновых вероятностей быстро становится очень малым и начинает сильно зависеть от длины ответа.

Ограничение методов VeriFree и RLPR для данной работы состоит в том, что они в первую очередь формулируются вокруг одного эталонного ответа или вокруг задач, где эталонный ответ можно считать достаточно определённым. Юмор устроен иначе. Один запрос может иметь множество хороших ответов, а наблюдаемое множество шуток является неполным и смещённым подмножеством возможных удачных вариантов. Поэтому прямое применение одноэталонной цели может ошибочно наказывать хорошие, но не наблюдавшиеся шутки, и чрезмерно усиливать отдельные формулировки из корпуса.

1.7 Позиционирование данной работы

Данная работа находится на пересечении трёх направлений: генерации юмора, обучения рассуждающих языковых моделей и обучения без внешнего верификатора. Литература по вычислительному юмору показывает, что задача является многомерной, контекстно зависимой и плохо сводится к единственной метрике [15, 1, 14, 11]. Работы по генерации юмора с рассуждением показывают, что промежуточный творческий поиск может улучшать ответы, но часто задают структуру этого поиска вручную [2, 25, 17, 10, 23]. Работы по обучению

без внешнего верификатора показывают, что вероятность эталонного ответа может служить внутренним сигналом обучения, но их исходная постановка плохо учитывает многореференсность и неполноту творческих задач [26, 22].

Основной исследовательский пробел состоит в том, что существующие методы либо рассматривают юмор как задачу генерации по вручную заданному сценарию рассуждения, либо рассматривают обучение рассуждений в доменах с объективной проверкой или одним эталонным ответом. В данной работе предлагается адаптировать идею обучения без внешнего верификатора к юмору как к частично наблюдаемой многореференсной задаче. Вместо единственного эталонного ответа используется множество эталонных шуток, найденных в корпусе по близости запроса. Вместо сырой вероятности полной шутки используется нормированная логарифмическая масса эталонных ответов, что уменьшает численные проблемы и смещение в пользу коротких последовательностей. Траектория рассуждения при этом остаётся порождаемой моделью, а не задаётся фиксированным шаблоном.

Такое позиционирование определяет вклад работы. Она не утверждает, что автоматическая оценка юмора полностью заменяет человеческую, и не предполагает, что наблюдаемые эталонные шутки исчерпывают все хорошие ответы. Напротив, работа рассматривает эти ограничения как часть постановки задачи. Цель состоит в разработке и экспериментальной проверке программного конвейера, который позволяет изучать, можно ли использовать многореференсную логарифмическую цель для обучения рассуждающей генерации юмора без отдельной модели награды.

Глава 2

Методология

2.1 Общая схема метода и программного конвейера

Методология работы состоит из двух связанных частей. Первая часть — программный конвейер построения данных, генерации кандидатов и автоматической оценки. Вторая часть — обучающая цель MRVF, то есть многореференсное обучение без внешнего верификатора, использующее найденные эталонные ответы как слабый сигнал для обучения траекторий рассуждения.

На высоком уровне система строит слабо размеченный набор данных с несколькими эталонными ответами из сырых корпусов шуток. Сначала шутки объединяются в единый корпус и очищаются от точных дублей и почти совпадающих текстов. Затем для каждой шутки строится плотное векторное представление, из текста извлекаются ключевые слова, а группы ключевых слов используются как слабые запросы для поиска семантически близких шуток. Полученные множества близких шуток образуют наблюдаемые наборы эталонных ответов $Y^*(x)$.

Эти эталонные множества используются в двух ролях. Во-первых, они задают контрольные запросы для генерации кандидатов и последующей автоматической оценки. Во-вторых, они входят непосредственно в обучающую функцию MRVF: модель получает запрос x , порождает траекторию рассуждения z , после чего эталонные ответы из $Y^*(x)$ оцениваются с помощью принудительного учительского декодирования. Таким образом, подготовка данных не является вспомогательным этапом: она определяет наблюдаемую структуру, относительно которой формулируется обучающая цель.

2.2 Построение набора данных

Набор данных объединяется из двух публичных коллекций шуток: Short Jokes и rJokes [19]. В репозитории этот этап реализован в `src/pipelines/jokes.py`. Конвейер скачивает сырые файлы из зафиксированных версий источников, преобразует их в формат Parquet и сохраняет метаданные источника для каждой шутки. Для каждой строки сохраняются поля `id`, `text`, `source_name`, `source_filename` и `source_id`. После отдельной предобработки двух корпусов они объединяются и получают новый глобальный целочисленный идентификатор.

Исходная коллекция содержит приблизительно 664 000 текстов шуток. После удаления точных дублей и почти совпадающих текстов размер текущего корпуса составляет приблизительно 570 000 шуток. Дедупликация важна не только как техническая очистка данных. В многореференсной постановке повторяющаяся шутка фактически получает больший вес в наблюдаемом множестве эталонных ответов, что может искусственно усиливать её вклад в обучающую цель. Поэтому удаление дублей является частью определения эмпирической меры над корпусом шуток.

2.2.1 Удаление дублей и почти совпадающих шуток

Для корпуса юмора полная агрессивная нормализация нежелательна: пунктуация, порядок слов и точная формулировка могут влиять на комический эффект. Поэтому очистка выполняется в несколько этапов. Сначала удаляются пустые записи и точные дубли после нормализации строки. Пусть $n(y)$ обозначает нормализованный текст шутки y , полученный после приведения Unicode к канонической форме, приведения к нижнему регистру и удаления неинформативных символов. Две шутки считаются точными дублями, если

$$n(y_i) = n(y_j).$$

Следующий уровень удаляет тексты, совпадающие по множеству значимых токенов. Пусть $T(y)$ — множество токенов шутки после простой токенизации и удаления служебных символов. Тогда токенный дубль определяется

условием

$$T(y_i) = T(y_j),$$

при условии, что число уникальных токенов достаточно велико. Этот дополнительный порог нужен, чтобы не удалять короткие тексты, которые случайно совпадают по одному или двум словам.

Наконец, для почти совпадающих шуток используются меры сходства по токенам и символьным фрагментам. Токенное сходство Жаккара определяется как

$$J_{\text{tok}}(y_i, y_j) = \frac{|T(y_i) \cap T(y_j)|}{|T(y_i) \cup T(y_j)|}.$$

Аналогично вычисляется сходство Жаккара по символьным шинглам J_{char} . Дополнительно используется нормированное сходство редактирования R_{edit} , оценивающее близость строк с учётом порядка символов. Пара шуток считается почти дублем, если соответствующие значения превышают заданные пороги:

$$J_{\text{tok}}(y_i, y_j) \geq \tau_{\text{tok}}, \quad J_{\text{char}}(y_i, y_j) \geq \tau_{\text{char}}, \quad R_{\text{edit}}(y_i, y_j) \geq \tau_{\text{edit}}.$$

Такой подход сохраняет консервативность очистки: корпус не сводится к абстрактным семантическим кластерам, но очевидные повторения и переформулировки одной и той же шутки удаляются. Это особенно важно для дальнейшего поиска эталонных ответов: если дубликаты оставить, то один и тот же юмористический ход будет многократно попадать в $Y^*(x)$ и завышать свою роль при обучении.

2.3 Плотный индекс векторных представлений

Следующий этап кодирует каждую шутку в плотном семантическом векторном пространстве. Он реализован в `src/pipelines/embeddings.py`. Текущая конфигурация использует модель семейства Qwen Embedding и хранит для каждой шутки вектор фиксированной размерности. Векторные представления записываются во фрагменты Parquet и хранятся как пары (i, e_i) , где i — глобальный идентификатор шутки, а $e_i \in \mathbb{R}^d$ — соответствующее представление.

Роль этого этапа двоякая. Во-первых, векторные представления используются для оценки релевантности ключевых слов-кандидатов. Во-вторых, они задают индекс поиска, с помощью которого для слабого запроса x извлекаются

семантически близкие шутки, образующие наблюдаемое множество эталонных ответов $Y^*(x)$.

Иными словами, пространство векторных представлений является общим представлением, от которого зависят и извлечение ключевых слов, и поиск эталонных ответов. Это важная инженерная особенность конвейера: качество многореференсной разметки зависит не от ручной разметки запросов, а от того, насколько хорошо векторное пространство группирует тематически и семантически близкие шутки.

2.4 Извлечение ключевых слов как поиск слабых запросов

Предполагается, что каждая шутка имеет набор слабых запросов, на которые она правдоподобно отвечает. Например, запрос “write a joke about a chicken” можно рассматривать как допустимый запрос для классической шутки “why did the chicken cross the road”. Этап извлечения ключевых слов направлен на построение таких слабых запросов на основе текста самой шутки. Он реализован в `src/pipelines/keywords.py`.

Для каждой шутки y_i процесс сначала генерирует пул n -грамм-кандидатов с помощью анализатора `CountVectorizer`. В конфигурации по умолчанию этот анализатор использует униграммы, удаляет английские стоп-слова и оставляет ограниченное число терминов-кандидатов на шутку. Пусть полученный набор кандидатов равен

$$C(y_i) = \{c_{i1}, \dots, c_{im}\}.$$

Затем каждый термин-кандидат кодируется той же моделью векторных представлений, что и шутки.

Если $e(y_i)$ обозначает векторное представление шутки, а $e(c)$ — векторное представление ключевого слова-кандидата, то релевантность кандидата оценивается через косинусное сходство:

$$s(c, y_i) = \frac{e(c)^\top e(y_i)}{\|e(c)\| \|e(y_i)\|}.$$

Выбор только наиболее похожих кандидатов часто давал бы избыточные ключевые слова. Например, несколько близких n -грамм могут описывать один и тот же объект или одну и ту же тему.

Для уменьшения избыточности применяется *maximal marginal relevance* (MMR), то есть процедура выбора кандидатов с балансом между релевантностью и разнообразием. На каждом шаге выбирается кандидат, который одновременно хорошо соответствует исходной шутке и не слишком похож на уже выбранные ключевые слова. В результате для шутки получается компактное представление

$$K(y_i) = \{k_{i1}, k_{i2}, k_{i3}\},$$

которое может содержать меньше трёх элементов, если пул кандидатов мал.

Методологически эти ключевые слова не следует интерпретировать как полностью восстановленные исходные запросы. Это более слабые объекты: короткие n -граммы, которые фиксируют тематические или лексические якоря шутки. Такое приближение не решает задачу восстановления запроса по шутке полностью, но даёт удобное представление, из которого можно строить контролируемые запросы для генерации.

2.5 Построение эталонных ответов через группировку ключевых слов и поиск

2.5.1 Группировка ключевых слов и формирование запросов

Этап построения эталонных ответов реализован в `src/pipelines/references`. Для каждого набора ключевых слов $K(y_i)$ процесс перечисляет все уникальные комбинации ключевых слов, размер которых лежит между `min_keywords` и `max_keywords`. В текущей постановке используются однословные и двухсловные группы.

Если

$$K(y_i) = \{k_1, k_2, k_3\},$$

то сгенерированные группы имеют вид

$$\{k_1\}, \{k_2\}, \{k_3\}, \{k_1, k_2\}, \{k_1, k_3\}, \{k_2, k_3\}.$$

Каждая группа преобразуется в запрос на естественном языке с помощью шаблона

`Write a joke using the following keyword(s): {keywords}`

Этот запрос служит двум целям. Во-первых, это текстовая форма, которая затем подаётся генеративным моделям. Во-вторых, этот же объект кодируется как поисковый запрос для извлечения эталонных ответов из корпуса.

Запросы с одним ключевым словом являются широкими, тематическими и часто допускают много возможных шуток. Запросы с двумя ключевыми словами более ограничены и обычно извлекают более точные кандидаты. Текущая реализация останавливается на двух ключевых словах, потому что более крупные группы быстро становятся слишком специфичными и уменьшают число правдоподобных различных эталонных ответов.

2.5.2 Поиск эталонных ответов с помощью индекса векторных представлений

После формирования строк запросов процесс кодирует их как поисковые запросы и извлекает близкие шутки из индекса FAISS, построенного по полной коллекции векторных представлений шуток. Перед индексацией и поиском векторы нормализуются, так что скалярное произведение соответствует косинусному сходству.

Для сформированного запроса x пусть $q(x)$ — его нормализованное векторное представление, а e_j — нормализованное векторное представление шутки y_j . Тогда оценка поиска равна

$$s(x, y_j) = q(x)^\top e_j.$$

Процедура поиска извлекает больше кандидатов, чем требуется в финальном наборе, фильтрует те, чьё сходство ниже заданного порога, и оставляет лучшие оставшиеся объекты. Соответствующие тексты используются как наблюдаемый

набор эталонных ответов:

$$Y^*(x) = \{y_1, \dots, y_n\}, \quad n \leq 5.$$

Важно, что $Y^*(x)$ не является полным множеством всех хороших шуток для запроса x . Это конечное множество найденных корпусных примеров, близких к запросу в выбранном векторном пространстве. Поэтому дальнейшая обучающая цель должна интерпретироваться как оптимизация по неполному наблюдаемому множеству эталонов, а не как оптимизация истинной человеческой смешности.

Этот этап реализует центральную идею работы: запрос связывается не с одним истинным ответом, а с несколькими шутками-кандидатами, которые могут служить эталонными ответами. Такая постановка лучше соответствует природе юмора, где один и тот же набор ключевых слов допускает множество разных удачных панчлайнов.

2.5.3 Дедупликация эталонных запросов и разбиение выборки

После поиска итоговый набор данных содержит строки вида

$$(x_i, Y^*(x_i), S(x_i)),$$

где x_i — запрос на основе ключевых слов, $Y^*(x_i)$ — найденный набор эталонных ответов, а $S(x_i)$ — вектор оценок поиска. Поскольку разные исходные шутки могут порождать одинаковые группы ключевых слов, набор эталонных запросов дополнительно дедуплицируется по полю **keywords**. При этом сохраняется первое вхождение, а идентификаторы строк назначаются заново.

Дедуплицированный набор затем разбивается на обучающую, валидационную и тестовую части. Разбиение выполняется случайно с фиксированным зерном, что обеспечивает воспроизводимость экспериментов. Отдельно сохраняются полный набор данных и каждая часть выборки. Такая организация позволяет использовать один и тот же набор эталонных запросов для обучения, генерации кандидатов и последующей автоматической оценки.

2.6 Генерация кандидатов

Этап генерации кандидатов поддерживает сравнение нескольких вариантов модели на одном и том же наборе запросов. Он реализован в `src/pipelines/candidates.py`. Для выбранной модели и части набора данных процесс проходит по запросам и вызывает интерфейс генерации ответов, совместимый с OpenAI API.

Используется тот же общий шаблон запроса, что и при построении эталонных ответов:

Write a joke using the following keyword(s): {keywords}

Конкретные экспериментальные запуски могут уточнять этот шаблон, например явно требовать вернуть только финальную шутку без объяснений. Такое уточнение важно, поскольку в задаче юмора формат ответа является частью качества: длинное объяснение или повторение инструкции может ухудшать оценку даже при наличии удачной идеи.

Схема выхода содержит четыре основных поля: идентификатор строки, группу ключевых слов, название модели и сгенерированный текст. Файлы с кандидатами сохраняются в структуре каталогов по модели и части выборки. Такая организация важна для последующей оценки, поскольку позволяет выравнивать ответы нескольких моделей по одному базовому идентификатору и запросу.

Методологически этот этап следует понимать как слой построения контрольных артефактов. Он создаёт ответы нескольких вариантов модели на одних и тех же запросах: базовых моделей, моделей в режиме явного рассуждения и моделей после MRVF-обучения. Благодаря общему идентификатору строки ответы разных моделей можно выравнивать по одному запросу и сравнивать попарно.

2.7 Оценка кандидатов

2.7.1 Попарное сравнение

Автоматическая оценка реализована в `src/pipelines/evaluation.py`. Процесс сначала объединяет наборы кандидатов от нескольких моделей. Затем

ответы группируются по идентификатору эталонного запроса и названию модели. Для каждого запроса строятся все попарные сравнения между моделями, которые сгенерировали непустые ответы.

Порядок левого и правого ответа рандомизируется с фиксированным зерном случайности, чтобы уменьшить позиционное смещение модели-оценщика. Оценочный запрос содержит исходный запрос на шутку, левый и правый кандидатные ответы. Системная инструкция просит модель-оценщик выбрать, какая шутка смешнее для широкой аудитории, избегать ничьих и возвращать строгий JSON-ответ. В текущем протоколе модель-оценщик используется при температуре 0.

Такой слой оценки измеряет только сравнительное качество ответов в рамках выбранной модели-оценщика. Он не заменяет человеческую оценку и не реализует многомерную рубрику, включающую новизну, релевантность, краткость или социальную уместность. Тем не менее попарное сравнение является удобным промежуточным инструментом, поскольку относительный выбор между двумя шутками обычно проще и устойчивее абсолютной числовой оценки.

2.7.2 Агрегация таблицы результатов

Попарные суждения агрегируются в рейтинг с числом побед, числом поражений, числом сравнений, долей побед и оценками Брэдли–Терри. Пусть $\mathcal{M}$ обозначает множество оцениваемых моделей. Для каждой пары моделей (m_a, m_b) и каждого запроса модель-оценщик выбирает победителя.

Модель Брэдли–Терри сопоставляет каждой модели m скрытый параметр качества q_m . Вероятность победы модели m_a над моделью m_b задаётся как

$$P(m_a \succ m_b) = \frac{\exp(q_{m_a})}{\exp(q_{m_a}) + \exp(q_{m_b})}.$$

В таблице результатов используется оценённое значение q_m , обозначаемое как `bt_score`. Такая агрегация позволяет получить единую относительную шкалу качества из набора попарных сравнений, но её следует интерпретировать только относительно выбранного набора моделей, запросов и модели-оценщика.

Таким образом, программный конвейер решает две задачи. С инженерной стороны он создаёт воспроизводимые артефакты: очищенный корпус, векторные представления, ключевые слова, эталонные множества, кандидатные

ответы и таблицы автоматической оценки. С методологической стороны он задаёт наблюдаемую структуру $Y^*(x)$, без которой невозможно сформулировать MRVF-обучение. Дальнейшие разделы описывают, как эти множества эталонных ответов входят в обучающую цель.

2.8 Теоретическая рамка обучения MRVF

Построенный выше конвейер связывает каждый запрос x с наблюдаемым множеством эталонных ответов $Y^*(x)$. Теперь опишем, как такая многореференсная разметка входит в обучающую цель. Центральная идея состоит в том, что генерация юмора плохо согласуется с предположением об одном правильном ответе. Слабый запрос может допускать много приемлемых шуток, различающихся завязкой, механизмом панчлайна, стилем и уровнем абсурдности. Поэтому обучающая цель должна учитывать не один эталонный ответ, а множество наблюдаемых эталонов.

2.8.1 Обозначения

Пусть $\mathcal{X}$, $\mathcal{Z}$ и $\mathcal{Y}$ обозначают пространства запросов, траекторий рассуждения и итоговых ответов соответственно. Для запроса $x \in \mathcal{X}$:

- $z \in \mathcal{Z}$ обозначает траекторию рассуждения;
- $y \in \mathcal{Y}$ обозначает итоговую сгенерированную шутку;
- $y^* \in \mathcal{Y}$ обозначает один эталонный ответ;
- $Y^*(x) \subset \mathcal{Y}$ обозначает наблюдаемое множество эталонных ответов, связанное с запросом x ;
- π_θ обозначает стратегию, то есть условную вероятностную модель над текстом, параметризованную θ .

Совместное распределение по траектории рассуждения и итоговому ответу записывается как

$$\pi_\theta(z, y \mid x) = \pi_\theta(z \mid x) \pi_\theta(y \mid x, z).$$

Когда модель явно выводит токены рассуждения, z можно отождествить с текстом внутри специального сегмента рассуждения, а y — с финальной шуткой. Если явное рассуждение отключено, ту же факторизацию можно понимать как концептуальное разложение поведения модели на выбор промежуточного состояния и последующую генерацию ответа.

При обсуждении неполноты будет полезно представлять более крупное скрытое множество

$$Y_{\text{true}}(x) \supseteq Y^*(x),$$

содержащее все действительно приемлемые шутки для запроса x . Это полное множество никогда не наблюдается на практике, но оно помогает формально описать ограничение многоореференсной разметки.

2.8.2 Обучение с верифицируемыми наградами

Обучение с подкреплением оптимизирует стратегию, максимизируя ожидаемую награду. В задачах с верифицируемым ответом награда может быть назначена внешней процедурой, не зависящей от субъективного человеческого суждения. Например, программу можно выполнить на наборе тестов, а итоговый ответ в математической задаче можно сравнить с каноническим решением. Именно такая структура лежит в основе ряда современных работ по обучению рассуждающих моделей [5, 12].

Сначала рассмотрим стандартную постановку с одним эталонным ответом. Для каждого запроса x существует проверяемая цель y^* , а награда задаётся как

$$R(y, y^*) = \mathbb{I}\{y = y^*\},$$

где $\mathbb{I}\{\cdot\}$ — индикаторная функция. Тогда целевая функция имеет вид

$$J_{\text{RLVR}}(\theta \mid x, y^*) = \mathbb{E}_{z \sim \pi_\theta(z \mid x)} \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x, z)} [R(y, y^*)].$$

Эквивалентно, в развернутой форме суммы:

$$J_{\text{RLVR}}(\theta \mid x, y^*) = \sum_{z \in \mathcal{Z}} \sum_{y \in \mathcal{Y}} \pi_\theta(z \mid x) \pi_\theta(y \mid x, z) R(y, y^*).$$

Для оптимизации этого математического ожидания используется тожде-

ство функции оценки, также известное как тождество REINFORCE:

$$\nabla_{\theta} \mathbb{E}_{u \sim p_{\theta}(u)} [f(u)] = \mathbb{E}_{u \sim p_{\theta}(u)} [f(u) \nabla_{\theta} \log p_{\theta}(u)].$$

Это тождество следует напрямую из соотношения

$$\nabla_{\theta} p_{\theta}(u) = p_{\theta}(u) \nabla_{\theta} \log p_{\theta}(u).$$

Применяя его к совместному распределению (z, y) , получаем

$$\begin{aligned} \nabla_{\theta} J_{\text{RLVR}} &= \nabla_{\theta} \sum_{z,y} \pi_{\theta}(z, y \mid x) R(y, y^*) \\ &= \sum_{z,y} R(y, y^*) \nabla_{\theta} \pi_{\theta}(z, y \mid x) \\ &= \sum_{z,y} \pi_{\theta}(z, y \mid x) R(y, y^*) \nabla_{\theta} \log \pi_{\theta}(z, y \mid x) \\ &= \mathbb{E}_{z,y} [R(y, y^*) \nabla_{\theta} \log \pi_{\theta}(z, y \mid x)]. \end{aligned}$$

Используя факторизацию совместной стратегии,

$$\log \pi_{\theta}(z, y \mid x) = \log \pi_{\theta}(z \mid x) + \log \pi_{\theta}(y \mid x, z),$$

получаем

$$\nabla_{\theta} J_{\text{RLVR}} = \mathbb{E}_{z,y} [R(y, y^*) (\nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} \log \pi_{\theta}(y \mid x, z))].$$

Это выражение имеет простую интерпретацию. Выбранная пара (z, y) получает награду только тогда, когда верификатор объявляет y правильным. Когда это происходит, обновление увеличивает и вероятность траектории рассуждения, приведшей к ответу, и вероятность самого ответа при этой траектории.

2.8.3 VeriFree

Юмор не предоставляет такого внешнего верификатора, и в более общем виде многие задачи открытой генерации не имеют детерминированного теста правильности. VeriFree закрывает этот разрыв, замечая, что в случае одного эталонного ответа ожидаемую награду точного совпадения можно переписать

как условную вероятность [26]. Зафиксируем x и z . Тогда

$$\begin{aligned}\mathbb{E}_{y \sim \pi_\theta(\cdot | x, z)}[\mathbb{I}\{y = y^*\}] &= \sum_{y \in \mathcal{Y}} \pi_\theta(y | x, z) \mathbb{I}\{y = y^*\} \\ &= \pi_\theta(y^* | x, z).\end{aligned}$$

Подставляя это тождество обратно в целевую функцию RLVR, получаем

$$J_{\text{RLVR}}(\theta | x, y^*) = \mathbb{E}_{z \sim \pi_\theta(z | x)}[\pi_\theta(y^* | x, z)].$$

Это мотивирует награду без внешнего верификатора

$$r_\theta(x, z, y^*) := \pi_\theta(y^* | x, z)$$

и соответствующую цель

$$J_{\text{VF}}(\theta | x, y^*) := \mathbb{E}_{z \sim \pi_\theta(z | x)}[r_\theta(x, z, y^*)].$$

Значимость такой переформулировки одновременно практическая и концептуальная. Для вычисления награды не требуется явное декодирование итогового ответа y . Вероятность $\pi_\theta(y^* | x, z)$ можно вычислить с помощью принудительного учительского декодирования на последовательность эталонного ответа, что удобно параллелизуется и убирает шум, связанный с выборкой финального ответа.

Чтобы получить градиент, запишем

$$J_{\text{VF}}(\theta | x, y^*) = \sum_{z \in \mathcal{Z}} \pi_\theta(z | x) r_\theta(x, z, y^*).$$

Дифференцируя по правилу произведения, получаем

$$\begin{aligned}\nabla_\theta J_{\text{VF}} &= \sum_z \left[\nabla_\theta \pi_\theta(z | x) r_\theta(x, z, y^*) + \pi_\theta(z | x) \nabla_\theta r_\theta(x, z, y^*) \right] \\ &= \sum_z \pi_\theta(z | x) \left[r_\theta(x, z, y^*) \nabla_\theta \log \pi_\theta(z | x) + \nabla_\theta r_\theta(x, z, y^*) \right].\end{aligned}$$

Так как $r_\theta(x, z, y^*) = \pi_\theta(y^* \mid x, z)$, имеем

$$\nabla_\theta r_\theta(x, z, y^*) = r_\theta(x, z, y^*) \nabla_\theta \log \pi_\theta(y^* \mid x, z).$$

Следовательно,

$$\nabla_\theta J_{\text{VF}} = \mathbb{E}_{z \sim \pi_\theta(z \mid x)} \left[r_\theta(x, z, y^*) (\nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta \log \pi_\theta(y^* \mid x, z)) \right].$$

Эта декомпозиция важна для дальнейшей работы. Первое слагаемое обновляет распределение траекторий рассуждения: оно повышает вероятность тех z , при которых сама модель назначает большую условную вероятность эталонному ответу. Второе слагаемое является прямым градиентом по эталонному ответу, вычисляемым с помощью принудительного учительского декодирования.

2.8.4 Многореференсное обобщение VeriFree

Предположение об одном эталонном ответе не подходит для генерации юмора. Запрос вида “напиши шутку с ключевыми словами cat и dog” не имеет единственного правильного продолжения: возможны разные завязки, разные панчлайны и разные стилистические решения. Поэтому награда должна задаваться не на одном эталонном ответе, а на наблюдаемом множестве допустимых ответов.

Пусть $Y^*(x) \subset \mathcal{Y}$ обозначает наблюдаемое множество эталонных ответов для запроса x . Определим многореференсную награду точного совпадения:

$$R(y, Y^*) = \mathbb{I}\{y \in Y^*\}.$$

Соответствующая целевая функция имеет вид

$$J_{\text{MR}}(\theta \mid x, Y^*) = \mathbb{E}_{z \sim \pi_\theta(z \mid x)} \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x, z)} [\mathbb{I}\{y \in Y^*\}].$$

Для фиксированных x и z

$$\begin{aligned}\mathbb{E}_{y \sim \pi_\theta(\cdot | x, z)} [\mathbb{I}\{y \in Y^*\}] &= \sum_{y \in \mathcal{Y}} \pi_\theta(y | x, z) \mathbb{I}\{y \in Y^*\} \\ &= \sum_{y \in Y^*} \pi_\theta(y | x, z).\end{aligned}$$

Это мотивирует многореференсную награду без внешнего верификатора:

$$r_\theta(x, z, Y^*) := \sum_{y \in Y^*} \pi_\theta(y | x, z).$$

Величина $r_\theta(x, z, Y^*)$ является вероятностной массой, которую модель при траектории z назначает наблюдаемому множеству эталонных ответов. Итоговая идеальная цель:

$$J_{\text{MRVF}}(\theta | x, Y^*) = \mathbb{E}_{z \sim \pi_\theta(z | x)} [r_\theta(x, z, Y^*)].$$

На уровне скалярной целевой функции MRVF играет для многореференсной награды ту же роль, что VeriFree играет для награды с одним точным ответом. Однако структура градиента меняется: прямое слагаемое теперь распределяется по нескольким эталонным ответам.

Действительно,

$$J_{\text{MRVF}}(\theta | x, Y^*) = \sum_z \pi_\theta(z | x) r_\theta(x, z, Y^*),$$

поэтому

$$\begin{aligned}\nabla_\theta J_{\text{MRVF}} &= \sum_z \left[\nabla_\theta \pi_\theta(z | x) r_\theta(x, z, Y^*) + \pi_\theta(z | x) \nabla_\theta r_\theta(x, z, Y^*) \right] \\ &= \mathbb{E}_{z \sim \pi_\theta(z | x)} \left[r_\theta(x, z, Y^*) \nabla_\theta \log \pi_\theta(z | x) + \nabla_\theta r_\theta(x, z, Y^*) \right].\end{aligned}$$

Теперь раскроем производную по эталонным ответам:

$$\begin{aligned}
\nabla_{\theta} r_{\theta}(x, z, Y^*) &= \nabla_{\theta} \sum_{y \in Y^*} \pi_{\theta}(y \mid x, z) \\
&= \sum_{y \in Y^*} \nabla_{\theta} \pi_{\theta}(y \mid x, z) \\
&= \sum_{y \in Y^*} \pi_{\theta}(y \mid x, z) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z).
\end{aligned}$$

Следовательно,

$$\nabla_{\theta} J_{\text{MRVF}} = \mathbb{E}_{z \sim \pi_{\theta}(z \mid x)} \left[r_{\theta}(x, z, Y^*) \nabla_{\theta} \log \pi_{\theta}(z \mid x) + \sum_{y \in Y^*} \pi_{\theta}(y \mid x, z) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z) \right].$$

Если $r_{\theta}(x, z, Y^*) > 0$, определим веса

$$w_{\theta}(y \mid x, z, Y^*) = \frac{\pi_{\theta}(y \mid x, z)}{r_{\theta}(x, z, Y^*)} \mathbb{I}\{y \in Y^*\}.$$

Тогда w_{θ} является распределением по наблюдаемому множеству эталонных ответов, и

$$\nabla_{\theta} r_{\theta}(x, z, Y^*) = r_{\theta}(x, z, Y^*) \sum_{y \in Y^*} w_{\theta}(y \mid x, z, Y^*) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z).$$

Эквивалентно,

$$\nabla_{\theta} \log r_{\theta}(x, z, Y^*) = \sum_{y \in Y^*} w_{\theta}(y \mid x, z, Y^*) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z).$$

Эта форма проясняет отличие от одноэталонного VeriFree. В VeriFree прямое слагаемое — это градиент $\log \pi_{\theta}(y^* \mid x, z)$ для одного эталонного ответа. В MRVF прямое слагаемое — это градиент логарифма суммы вероятностей по многим эталонным ответам. Поэтому обновление не отдаёт априорного предпочтения одному эталону; оно увеличивает суммарную вероятностную массу на наблюдаемом множестве, причём каждый эталонный ответ взвешивается текущей вероятностной массой стратегии, назначенной ему.

2.8.5 Логарифмическая масса как практическая цель

Идеальная формулировка MRVF использует вероятности полных последовательностей:

$$\pi_{\theta}(y \mid x, z) = \prod_{t=1}^{L_y} \pi_{\theta}(y_t \mid x, z, y_{<t}).$$

Для коротких проверяемых ответов такая величина может быть удобной. Для шуток она становится проблематичной: полная шутка обычно состоит из десятков токенов, поэтому произведение токеновых вероятностей быстро становится очень малым. Кроме того, ненормированная вероятность последовательности систематически предпочитает более короткие ответы, поскольку каждый дополнительный токен умножает вероятность на число меньше единицы.

Поэтому в практической реализации используется логарифмическая суррогатная цель. Для эталонного ответа y_j^* при запросе x и траектории рассуждения z_i сначала вычисляется токеновое логарифмическое правдоподобие:

$$\ell_{ij} = \sum_{t=1}^{L_j} \log \pi_{\theta}(y_{j,t}^* \mid x, z_i, y_{j,<t}^*).$$

Чтобы уменьшить смещение в пользу коротких эталонных ответов, используется нормировка по длине:

$$s_{ij} = \frac{1}{L_j^{\gamma}} \ell_{ij}.$$

В основной реализации используется среднее логарифмическое правдоподобие на токен, то есть $\gamma = 1$:

$$s_{ij} = \frac{1}{L_j} \sum_{t=1}^{L_j} \log \pi_{\theta}(y_{j,t}^* \mid x, z_i, y_{j,<t}^*).$$

Затем для траектории z_i вычисляется многореференсная логарифмическая масса:

$$m_i = \log \sum_{j=1}^M \alpha_j \exp(s_{ij}),$$

где $M = |Y^*(x)|$, а α_j — вес j -го эталонного ответа. В базовой реализации используются равные веса $\alpha_j = 1/M$. Операция $\log \sum \exp$ обеспечивает числен-

ную устойчивость и позволяет агрегировать несколько эталонных ответов без явного перехода к крайне малым вероятностям полных последовательностей.

Таким образом, идеальная цель MRVF мотивирует суммирование вероятностной массы по множеству эталонных ответов, а реализованная цель использует её устойчивую логарифмическую аппроксимацию. Это изменение не является чисто технической деталью: оно меняет масштаб и геометрию оптимизации. Однако для длинных открытых ответов такая замена необходима, поскольку сырые вероятности последовательностей дают слишком слабый и сильно зависящий от длины сигнал.

2.8.6 Неполнота наблюдаемых эталонных ответов

Наблюдаемое множество $Y^*(x)$ не является полным множеством всех хороших шуток для запроса x . Полезно мысленно различать $Y^*(x)$ и скрытое множество $Y_{\text{true}}(x)$, содержащее все ответы, которые были бы признаны удачными человеческой аудиторией:

$$Y^*(x) \subseteq Y_{\text{true}}(x).$$

Множество $Y_{\text{true}}(x)$ никогда не наблюдается полностью. Это принципиальное отличие юмора от задач с проверяемым ответом.

Возникают две формы неполноты. Первая — структурная. Даже если для одного запроса найдено несколько эталонных шуток, существует много других допустимых панчлайнов, отсутствующих в корпусе или не найденных индексом. Вторая — вычислительная. Даже внутри наблюдаемого множества $Y^*(x)$ на практике может использоваться только ограниченное число эталонных ответов, чтобы уменьшить стоимость принудительного учительского декодирования.

Эта неполнота создаёт риск ложных отрицательных примеров. Оптимизация m_i или $r_\theta(x, z, Y^*)$ увеличивает массу на наблюдаемые эталоны. Поскольку распределение модели по всем возможным ответам нормировано,

$$\sum_{y \in \mathcal{Y}} \pi_\theta(y \mid x, z) = 1,$$

увеличение массы на $Y^*(x)$ может происходить за счёт ответов из $\mathcal{Y} \setminus Y^*(x)$. Но это дополнение содержит не только плохие шутки, но и хорошие, не попавшие

в наблюдаемое множество:

$$Y_{\text{true}}(x) \setminus Y^*(x).$$

Следовательно, MRVF-обучение по неполному множеству эталонов может непреднамеренно подавлять допустимые, но ненаблюдаемые юмористические продолжения.

Этот риск не является ошибкой вывода градиента. Он является структурным следствием обучения по неполной разметке в области с большим числом допустимых решений. Поэтому результаты MRVF следует интерпретировать не как прямую оптимизацию истинной человеческой смешности, а как оптимизацию наблюдаемой многореференсной суррогатной цели. В экспериментальной части эта проблема учитывается через сравнение с базовыми моделями и автоматическую внешнюю оценку сгенерированных ответов.

2.8.7 Использование подмножества эталонных ответов

Если наблюдаемое множество $Y^*(x)$ велико, вычисление правдоподобия всех эталонных ответов для каждой траектории z_i может быть дорогим. Поэтому на практике можно использовать подмножество $Y \subseteq Y^*(x)$. Для идеальной вероятностной массы

$$r_\theta(x, z, Y^*) = \sum_{y \in Y^*} \pi_\theta(y \mid x, z)$$

случайное равномерное подмножество даёт простую связь с полной величиной. Пусть каждый эталонный ответ включается в Y с вероятностью ρ . Тогда

$$r_\theta(x, z, Y) = \sum_{y \in Y^*} \mathbb{I}\{y \in Y\} \pi_\theta(y \mid x, z),$$

и

$$\mathbb{E}_Y[r_\theta(x, z, Y)] = \rho r_\theta(x, z, Y^*).$$

Следовательно, для сырой вероятностной массы случайное подмножество даёт оценку, пропорциональную полной массе в математическом ожидании.

Для логарифмической массы ситуация сложнее. Величина

$$m(x, z, Y) = \log \sum_{y \in Y} \alpha_y \exp(s_\theta(y \mid x, z))$$

не является несмещённой оценкой $m(x, z, Y^*)$, поскольку логарифм стоит после суммирования. Поэтому использование подмножества в реализованной цели следует понимать как стохастическое приближение для снижения вычислительной стоимости, а не как строго несмещённый оценщик полной логарифмической массы. Это ещё одна причина рассматривать MRVF как суррогатную обучающую цель, а не как точную оптимизацию истинного множества всех удачных шуток.

2.8.8 Цикл обучения

Для каждого запроса x модель выбирает группу из G траекторий рассуждения:

$$z_1, \dots, z_G \sim \pi_\theta(z \mid x).$$

Для каждой траектории z_i вычисляются нормированные логарифмические правдоподобия эталонных ответов s_{ij} , после чего агрегируется многореференсная логарифмическая масса

$$m_i = \log \sum_{j=1}^M \alpha_j \exp(s_{ij}).$$

Слагаемое, отвечающее за выбор траектории рассуждения, имеет вид оценщика функции оценки. Его задача — повысить вероятность тех траекторий z_i , для которых наблюдаемое множество эталонных ответов получает большую логарифмическую массу. Для уменьшения зависимости от масштаба награды внутри одного запроса используется групповая нормировка, аналогичная GRPO:

$$A_i = \frac{m_i - \mu_x}{\sigma_x + \varepsilon}, \quad \mu_x = \frac{1}{G} \sum_{k=1}^G m_k,$$

$$\sigma_x = \sqrt{\frac{1}{G} \sum_{k=1}^G (m_k - \mu_x)^2}.$$

Тогда слагаемое для обновления распределения траекторий записывается как

$$\mathcal{L}_z = -\frac{1}{G} \sum_{i=1}^G \text{sg}(A_i) \log \pi_{\theta}(z_i | x),$$

где $\text{sg}(\cdot)$ обозначает остановку градиента.

Остановка градиента здесь принципиальна. Преимущество A_i используется как коэффициент оценителя функции оценки, а не как дифференцируемая часть вычислительного графа. Если пропустить градиент через A_i , автоматическое дифференцирование добавит дополнительные слагаемые, не соответствующие исходному выводу. Прямой градиент по эталонным ответам учитывается отдельно, через слагаемое логарифмической массы.

Прямое эталонное слагаемое задаётся как отрицательная логарифмическая масса:

$$\mathcal{L}_{\text{ref}} = -\frac{1}{G} \sum_{i=1}^G m_i.$$

Оно отвечает за увеличение условного правдоподобия эталонных ответов при уже выбранных траекториях рассуждения. В раскрытой форме это слагаемое соответствует градиенту

$$\nabla_{\theta} m_i = \sum_{j=1}^M \rho_{ij} \nabla_{\theta} s_{ij},$$

где

$$\rho_{ij} = \frac{\alpha_j \exp(s_{ij})}{\sum_{k=1}^M \alpha_k \exp(s_{ik})}.$$

Таким образом, эталонные ответы внутри $Y^*(x)$ получают веса, пропорциональные их текущему нормированному правдоподобию при данной траектории z_i . Если несколько эталонов имеют сопоставимую массу, градиент распределяется между ними; если один эталон доминирует, обновление концентрируется на нём.

Итоговая функция потерь для одного запроса имеет вид

$$\mathcal{L}_{\text{MRVF}} = \lambda_z \mathcal{L}_z + \lambda_{\text{ref}} \mathcal{L}_{\text{ref}},$$

где λ_z и λ_{ref} управляют относительным вкладом обновления траекторий рассуждения и прямого эталонного слагаемого. В мини-пакете эта величина усред-

няется по запросам.

Эта формулировка сохраняет основную идею VeriFree: награда получается не от внешней модели-верификатора, а из вероятностей самой обучаемой модели. Отличие состоит в двух изменениях, необходимых для юмора. Во-первых, используется множество эталонных ответов вместо одного. Во-вторых, сырая вероятность полной последовательности заменяется нормированной логарифмической массой, более устойчивой для длинных и вариативных ответов.

Связь с RLOO. Альтернативным способом снижения дисперсии является базовый уровень с исключением текущего элемента:

$$b_i = \frac{1}{G-1} \sum_{k \neq i} m_k, \quad A_i^{\text{RLOO}} = m_i - b_i.$$

Такой оценщик ближе к классическому выводу REINFORCE, поскольку вычитает базовый уровень, не зависящий от текущей траектории z_i . Групповая стандартизация, используемая в GRPO,

$$A_i = \frac{m_i - \mu_x}{\sigma_x + \varepsilon},$$

является более сильной инженерной нормировкой: она не только центрирует, но и масштабирует значения внутри группы. Это помогает стабилизировать обучение при разных запросах, однако меняет точный масштаб оценщика. В данной работе эта нормировка рассматривается как практическая суррогатная процедура, выбранная для устойчивости обучения.

2.8.9 Замечание о базовых уровнях и остановке градиента

Механизмы снижения дисперсии применимы прежде всего к слагаемому функции оценки по траектории z . Если $b(x)$ не зависит от выбранной траекто-

рии z_i , то

$$\begin{aligned}\mathbb{E}_{z_i \sim \pi_\theta(\cdot | x)} [b(x) \nabla_\theta \log \pi_\theta(z_i | x)] &= b(x) \sum_{z_i} \pi_\theta(z_i | x) \nabla_\theta \log \pi_\theta(z_i | x) \\ &= b(x) \sum_{z_i} \nabla_\theta \pi_\theta(z_i | x) \\ &= b(x) \nabla_\theta 1 = 0.\end{aligned}$$

Поэтому вычитание базового уровня из награды не меняет математическое ожидание слагаемого функции оценки, но может уменьшить его дисперсию.

Прямое эталонное слагаемое устроено иначе. Оно не является оценщиком по выбранному ответу $y \sim \pi_\theta(\cdot | x, z)$, а вычисляется через принудительное учительское декодирование наблюдаемых эталонных ответов. Поэтому наивное вычитание базового уровня из $\mathcal{L}_{\text{ref}}$ в общем случае вносит смещение.

Например, в случае одного эталонного ответа выражение

$$\mathbb{E}_z \left[(r_\theta(x, z, y^*) - b(x)) \nabla_\theta \log \pi_\theta(y^* | x, z) \right]$$

раскладывается как

$$\mathbb{E}_z \left[r_\theta(x, z, y^*) \nabla_\theta \log \pi_\theta(y^* | x, z) \right] - b(x) \mathbb{E}_z \left[\nabla_\theta \log \pi_\theta(y^* | x, z) \right].$$

Второй член в общем случае не равен нулю. Та же проблема сохраняется в многокореференсном случае с градиентом логарифмической массы. Поэтому групповая нормировка используется только в слагаемом $\mathcal{L}_z$, а прямое эталонное слагаемое вычисляется непосредственно из логарифмической массы.

Та же логика объясняет использование оператора sg в $\mathcal{L}_z$. Значение преимущества используется как числовой коэффициент при $\log \pi_\theta(z_i | x)$. Дифференцировать этот коэффициент внутри слагаемого функции оценки не нужно: соответствующий градиент по эталонным ответам уже представлен отдельным слагаемым $\mathcal{L}_{\text{ref}}$.

В совокупности MRVF наследует ключевое вычислительное преимущество обучения без внешнего верификатора: он избегает внешней модели награды и явной выборки итоговых ответов для проверки. Однако для юмора метод должен быть адаптирован к многокореференсному и неполному характеру задачи. Поэтому центральными элементами предлагаемого подхода становятся

построение наблюдаемого множества $Y^*(x)$, нормированная логарифмическая масса эталонных ответов и групповая нормировка траекторий рассуждения.

Глава 3

Эксперименты

3.1 Цель экспериментальной части

Экспериментальная часть работы проверяет не только качество итоговых шуток, но и работоспособность всего предложенного конвейера. В частности, эксперименты должны ответить на четыре вопроса:

1. можно ли обучать LoRA-адаптеры для MRVF-цели на моделях Qwen3-1.7B и Qwen3-4B;
2. как MRVF-модели сравниваются с соответствующими базовыми моделями;
3. влияет ли режим явного рассуждения на качество короткой юмористической генерации;
4. согласуется ли оптимизация внутренней многореференсной цели с внешней автоматической попарной оценкой.

Эксперименты в данной работе являются предварительными: они предназначены для проверки реализуемости метода и выявления основных проблем постановки. Поэтому результаты следует интерпретировать как диагностические, а не как окончательное утверждение о превосходстве или несостоятельности MRVF для генерации юмора.

3.2 Используемые данные

Во всех экспериментах используется набор данных `halaction/humor-generation`. Исходная коллекция содержала приблизительно 664 000 текстов шуток. После

удаления точных дублей и почти совпадающих текстов текущий корпус содержит приблизительно 570 000 шуток. На его основе были построены векторные представления, группы ключевых слов и многореференсные запросы.

Для обучения MRVF использовалась конфигурация данных **references**. Каждая строка этого набора содержит запрос, построенный из ключевых слов, и несколько найденных эталонных шуток. Обучение проводилось на части **train**, а генерация и оценка кандидатов — на части **validation**. Разбиение было зафиксировано заранее, чтобы одни и те же запросы могли использоваться для сравнения разных моделей.

Основные объёмные характеристики данных приведены в Таблице 3.1.

Таблица 3.1: Объёмные характеристики данных, использованных в экспериментах

Компонент	Количество
Исходная коллекция шуток	$\approx 664\,000$
Корпус после дедупликации	$\approx 570\,000$
Группы ключевых слов	$\approx 1\,500$
Многореференсные запросы	$\approx 14\,600$
Максимальное число эталонов на запрос	5
Число эталонов в обучении на один запрос	2

3.3 Обучаемые модели

В экспериментах использовались две базовые языковые модели семейства Qwen3:

- Qwen/Qwen3-1.7B;
- Qwen/Qwen3-4B.

Для обеих моделей обучались LoRA-адаптеры, а не все параметры базовой модели. Такой выбор снижает требования к памяти и делает возможными отладочные запуски на одной видеокарте A100. В обоих случаях обучение выполнялось в формате bfloat16 с включённой экономией памяти через gradient checkpointing.

Основные параметры обучения приведены в Таблице 3.2. В таблицу включены только параметры, влияющие на постановку эксперимента; дополнительные инженерные параметры сохранены в конфигурационных файлах проекта.

Таблица 3.2: Основные параметры MRVF-обучения

Параметр	Qwen3-1.7B	Qwen3-4B
Число шагов обучения	80	35
Скорость обучения	$7 \cdot 10^{-6}$	$5 \cdot 10^{-6}$
Размер группы G	2	2
Число эталонов на запрос	2	2
Макс. длина рассуждения	512	384
Макс. длина эталонного ответа	64	64
Целевая функция	log-масса	log-масса
Нормировка эталонного ответа	среднее по токенам	среднее по токенам
Нормировка преимуществ	GRPO	GRPO
λ_z	1.0	1.0
λ_{ref}	0.1	0.1
LoRA rank	32	16
LoRA alpha	64	32
Тип чисел	bfloat16	bfloat16

Для Qwen3-1.7B использовалась конфигурация `qwen3-17b-free-think-r5-bud`. Для Qwen3-4B использовалась конфигурация `qwen3-4b-free-think-r1-budgeted.y`. Обе конфигурации используют режим рассуждения Qwen, принудительное закрытие блока рассуждения и MRVF-цель на основе нормированной логарифмической массы.

3.4 Сравнимые варианты моделей

Для каждой базовой модели сравнивались четыре режима:

1. базовая модель без явного рассуждения;
2. базовая модель с явным рассуждением;
3. MRVF-модель с явным рассуждением;
4. MRVF-модель без явного рассуждения.

Такой дизайн нужен, чтобы отделить эффект обучения от эффекта режима рассуждения. Если сравнивать только базовую модель без рассуждения и MRVF-модель с рассуждением, невозможно понять, связано ли изменение качества с обучением или с другим режимом вывода. Поэтому для каждой размерности модели оцениваются все четыре сочетания.

Список сравниваемых вариантов приведён в Таблице 3.3.

Таблица 3.3: Сравнимые варианты моделей

Группа	Обозначение	Обучение	Рассужд
1.7B	qwen3-17b-base-nothink-r5eval	нет	нет
1.7B	qwen3-17b-base-think-r5eval	нет	да
1.7B	qwen3-17b-mrvf-free-think-r5-budgeted	MRVF	да
1.7B	qwen3-17b-mrvf-free-think-r5-budgeted-nothink	MRVF	нет
4B	qwen3-4b-base-nothink	нет	нет
4B	qwen3-4b-base-think	нет	да
4B	qwen3-4b-mrvf-r1-budgeted-think	MRVF	да
4B	qwen3-4b-mrvf-r1-budgeted-nothink	MRVF	нет

3.5 Протокол генерации кандидатов

Кандидатные ответы генерировались на валидационной части набора **references**. Для каждого запроса модель должна была вернуть одну короткую шутку по заданным ключевым словам. В режимах без рассуждения генерация ограничивалась коротким ответом. В режимах с рассуждением модель сначала получала отдельный бюджет на блок рассуждения, после чего генерировала короткий финальный ответ.

Для Qwen3-1.7B использовалось 200 валидационных запросов. После фильтрации непустых ответов, общих для всех четырёх вариантов, осталось 184 запроса. Для Qwen3-4B использовалось 100 валидационных запросов. Параметры генерации приведены в Таблице 3.4.

Таблица 3.4: Параметры генерации кандидатных ответов

Группа	Режим	Бюджет рассуждения	Бюджет финального ответа
1.7B	без рассуждения	—	48 токенов
1.7B	с рассуждением	2048 токенов	48 токенов
4B	без рассуждения	—	48 токенов
4B	с рассуждением	1536 токенов	48 токенов

Во всех режимах с рассуждением использовалось принудительное закрытие блока рассуждения перед генерацией финального ответа. Это было необходимо для того, чтобы отделить внутреннее рассуждение модели от текста, который затем сравнивается моделью-оценщиком.

3.6 Протокол автоматической оценки

Автоматическая оценка проводилась попарно. Для каждого запроса и каждой пары моделей формировалось сравнение двух ответов. Порядок левого и правого ответа случайно перемешивался с фиксированным зерном случайности, чтобы уменьшить позиционное смещение. Модель-оценщик получала исходный запрос и два кандидатных ответа, после чего выбирала более удачную шутку.

Для четырёх моделей число пар моделей равно

$$\binom{4}{2} = 6.$$

Поэтому для N общих запросов число попарных сравнений равно

$$6N.$$

В эксперименте с Qwen3-1.7B использовалось $N = 184$ общих непустых запросов, что даёт 1104 сравнения. В эксперименте с Qwen3-4B использовалось $N = 100$ запросов, что даёт 600 сравнений. Эти объёмы приведены в Таблице 3.5.

Таблица 3.5: Объём автоматической попарной оценки

Группа	Число моделей	Число запросов	Число сравнений
Qwen3-1.7B	4	184	1104
Qwen3-4B	4	100	600

По результатам попарных сравнений для каждой модели вычислялись число побед, число поражений, доля побед и оценка Брэдли–Терри. Эти величины используются в следующей главе для представления итоговых результатов.

3.7 Сохранение артефактов и воспроизводимость

Для воспроизводимости экспериментов сохранялись несколько типов артефактов:

- конфигурации обучения;
- логи обучения и выборки промежуточных ответов;
- LoRA-адаптеры обученных моделей;
- сгенерированные кандидатные ответы;
- результаты попарной оценки;
- агрегированные таблицы лидеров.

LoRA-адаптеры были опубликованы отдельно от базовых моделей в репозиториях `halaction/Qwen3-1.7B-humor-lora` и `halaction/Qwen3-4B-humor-lora`. Кандидатные ответы, результаты оценки и таблицы лидеров были добавлены в набор данных `halaction/humor-generation`. Такая структура позволяет отдельно воспроизводить обучение, генерацию, оценку и агрегацию результатов.

3.8 Статус текущего экспериментального прогона

На момент написания этой версии текста был выполнен полный тестовый прогон: обучение MRVF-адаптеров, генерация кандидатов, загрузка контрольных точек, попарная оценка и построение таблиц лидеров. Однако этот прогон не рассматривается как окончательная оценка качества моделей.

После анализа сгенерированных ответов была обнаружена проблема в шаблоне генерации: часть ответов содержала пояснения, повторяющиеся фрагменты или не отделяла финальную шутку от рассуждения. Для задачи короткой генерации юмора это существенно искажает сравнение, поскольку модель-оценщик сравнивает не только юмористическую идею, но и формат ответа. По-

этому текущий прогон используется как проверка работоспособности конвейера, а не как финальный результат по качеству.

После исправления шаблона генерации кандидаты и попарная оценка должны быть пересчитаны. В следующей главе таблицы результатов представлены в структуре, рассчитанной на замену предварительных значений окончательными после повторного прогона.

Глава 4

Результаты

4.1 Текущие результаты

Реализованный процесс уже поддерживает сбор корпуса, генерацию векторных представлений, извлечение ключевых слов, построение запросы, поиск эталонных ответов, генерацию кандидатов и автоматическая попарная оценка. Оставшийся отсутствующий этап — собственно обучения с подкреплением обновление цикл. Поэтому данный раздел является узким: он описывает завершенные артефакты данных и фиксирует конкретный экспериментальный постановка для первых MRVF обучающие запуски.

Результат на уровне корпуса — объединенная коллекция шуток примерно из 664k объектов. В финальной схема обучения этот полный корпус служит пул поиска: все шутки кодируются векторных представлений и индексируются, а эталонные ответы для слабые запросы извлекаются из этого полного пространства векторных представлений. Однако извлечение ключевых слов нужно применять только к достаточно большому подмножеству, чтобы достичь целевого объем выборки. В нашем случае 3 — 5k группы ключевых слов могут дать 10 — 20k эталонные ответы для обучение, валидация и тестирование.

Для предварительный этап этого достаточно. Важный результат состоит в том, что данные и экспериментальный постановка теперь финализированы: корпус существует, эталонный ответ коллекция реализована, методология зафиксирована, а оставшаяся работа ограничена цикл обучения MRVF.

4.2 Планируемая процедура обучения

Репозиторий уже реализует полный процесс подготовки данных на Python с использованием Hugging Face Наборы данных, Parquet артефакты, FAISS-

based ближайших соседей поиск и OpenAI-compatible API вызовы для векторное представление генерация, генерацию кандидатов и автоматический оценка. Однако для собственно обучающие запуски стек программного обеспечения немного изменится.

Планируемая реализация будет использовать модифицированную версию **trl** для оптимизация и **vLLM** для генерация. **trl** предоставляет со стороны обучения логика для оптимизация стратегии, тогда как **vLLM** будет использоваться и для генерации ответы базовых моделей, и для обслуживания траектории генерации во время обучение с подкреплением. Поскольку MRVF зависит от выборка нескольких траектории рассуждения на запрос, пропускная способность генерации траекторий важен для эффективности.

Целевые модели относятся к семейству Qwen3 в диапазоне размеров 1.7B и 4B. Для каждого размера планируются следующие базовые модели:

1. **Qwen3 Instruct**, базовая модель в без режима рассуждения режим;
2. **Qwen3 Thinking**, базовая модель в thinking режим;
3. **MRVF-обученных Qwen3 Thinking**, предлагаемая модель;

Такая постановка сохраняет интерпретируемость сравнения. Он изолирует три отдельных вопроса: достаточно ли обычной инструкция настройка для генерации юмора; улучшает ли родной thinking режим генерацию юмора уже без дополнительного обучения; и может ли MRVF дополнительно улучшить thinking модель.

Все обучающие запуски планируются на 1–2 NVIDIA H200 GPUs в HSE НРС кластер. Этого достаточно для целевых размеров моделей и позволяет распараллеливать эксперименты между объекты.

Сравнение будет использовать процесс оценки, уже реализованный в репозитории. Ответы сопоставляются для одного запроса, порядок левого и правого вариантов рандомизируется, оценщик принуждается выбрать ровно одного победитель, а оценки моделей агрегируются через Bradley-Terry рейтинг. Этого достаточно для эксперимента, поскольку цель состоит в обнаружении того, дает ли MRVF измеримый сдвиг относительно instruct и базовые модели в режиме рассуждения. Более крупная многомерный оценка с оценивание по рубрике и человеческую разметку планируется после завершения обучения.

4.3 Планируемые абляции

План абляций напрямую связан с теорией. Наиболее информативные абляции следующие:

- **Число наблюдаемых эталонных ответов на запрос $|Y^*|$.** Сравнить меньшие и большие наборы эталонных ответов, например через обрезку результатов поиска до 1, 3 или 5 объектов.
- **коэффициент KL β .** Проверить, насколько сильно модель должна быть удерживаться рядом с начальной контрольной точкой, чтобы компенсировать неполнота.
- **Размер группы G .** Варьировать число выбранный траектории рассуждения на один запрос, чтобы оценить variance / пропускная способность компромисс MRVF оценщик.
- **Размер модели.** Сравнить 1.7B и 4B, а также проверить, насколько MRVF устойчив при применении к меньшим моделям.

Заключение

Данная работа рассматривала генерацию юмора как задачу рассуждения в неверифицируемом домене. Центральное утверждение состояло в том, что обучение юмору должно отходить от имитации одного референса и переходить к распределению вероятностной массы по множеству эталонных ответов при слабых запросах. Обзор литературы показал, почему такой сдвиг необходим: SFT склонен к имитации и запоминанию, основанный на предпочтениях RL нестабилен в юморе из-за шумных оценок, а современные многошаговые системы генерации юмора повышают качество ценой жестко заданных структур рассуждения.

Работа внесла два связанных вклада. На методологическом уровне она сформулировала расширение обучения с подкреплением без внешнего верификатора на случай нескольких эталонных ответов, в котором награда задается полной вероятностной массой, назначенной множеству приемлемых эталонных ответов, а не одному золотому ответу. На уровне реализации она построила и проанализировала пилотный процесс, который преобразует большой корпус шуток в запрос-эталонный ответ наборы через векторных представлений, извлечение ключевых слов, векторный поиск, удаление дублей и разбиение данных.

Эмпирический анализ пилота показал, что идея является практически осуществимой. В текущем локальном состоянии репозитория уже собран корпус из 664,048 шуток, пилотное подмножество было встроенный и обработано, а итоговый рабочий процесс производит 5,058 дедулицированных запрос-эталонный ответ строк со средним числом эталонных ответов 4.07 на запрос и медианой пять. Эти статистики показывают, что ключевое структурное требование обучения MRVF, а именно наличие запросы с несколькими приемлемыми эталонными ответами, может быть удовлетворено на практике.

В то же время работа остается предварительной в корректном смысле. Процедура построения запросы все еще приближенная, последующий артефакты контрольного набора еще не закоммичены, а сам цикл обучения MRVF пока

не реализован. Поэтому данную работу следует читать как основу дипломного исследования, а не как окончательный эмпирический вердикт об обучении с подкреплением для юмора.

Главный вывод состоит в том, что проект уже перешел границу от спекулятивного отчета к защищаемой исследовательской программе. Представление данных, программный процесс и формальная целевая функция согласованы друг с другом. Следующий шаг состоит в реализации этапа обучения и проверке того, сможет ли построенный набор данных с несколькими эталонными ответами действительно вызывать более качественное рассуждение и более сильный юмор, чем стандартные базовые методы постобучения.

Список литературы

- [1] Mostafa Amin и Manuel Burghardt. «A survey on approaches to computational humor generation». В: *Proceedings of the 4th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*. 2020, с. 29—41. URL: <https://aclanthology.org/2020.latechclfl-1.4/>.
- [2] Yahui Chen, Buxin Shi и Meng Si. *Prompt to GPT-3: Step-by-step thinking instructions for humor generation*. arXiv:2306.13195. 2023. URL: <https://arxiv.org/abs/2306.13195>.
- [3] Tianyu Chu, Shuhua Zhang, Yu Chen, Ning Ding и Tao Yu. *SFT memorizes, RL generalizes: A comparative study of foundation model post-training*. OpenReview / arXiv preprint. 2025. URL: <https://openreview.net/forum?id=2P4rYTtGok>.
- [4] Andrea Cocchieri, Lorenzo Ragazzi, Pietro Italiani, Giuseppe Tagliavini и Gianluca Moro. «“What do you call a dog that is incontrovertibly true? Dogma”: Testing LLM generalization through humor». В: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025, с. 22922—22937. URL: <https://doi.org/10.18653/v1/2025.acl-long.1117>.
- [5] DeepSeek-AI. *DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning*. arXiv:2501.12948. 2025. URL: <https://doi.org/10.48550/arXiv.2501.12948>.
- [6] Leo Gao, John Schulman, Jacob Hilton и др. *Scaling laws for reward model overoptimization*. arXiv:2210.10760. 2023. URL: <https://arxiv.org/abs/2210.10760>.
- [7] Akul Gunjal, Alex Wang, Esther Lau, Vedaant Nath, Yichao He, Bo Liu и Sean Hendryx. *Rubrics as rewards: Reinforcement learning beyond verifiable*

- domains*. arXiv:2507.17746. 2025. URL: <https://doi.org/10.48550/arXiv.2507.17746>.
- [8] Hamid Hashemi, Jason Eisner, Corby Rosset, Benjamin Van Durme и Chris Kedzie. «LLM-Rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts». B: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024. URL: <https://aclanthology.org/2024.acl-long.745/>.
 - [9] Justine T. Kao, Roger Levy и Noah D. Goodman. «A computational model of linguistic humor in puns». B: *Cognitive Science* 40.5 (2016), с. 1270—1285. URL: <https://doi.org/10.1111/cogs.12269>.
 - [10] Sangho Kim и Lydia B. Chilton. *AI humor generation: Cognitive, social and creative skills for effective humor*. arXiv:2502.07981. 2025. URL: <https://arxiv.org/abs/2502.07981>.
 - [11] Jens Lemmens и Victor De Marez. «Computational humor modeling: A survey on the state of the art». B: *ACM Computing Surveys* 58.7 (2026), 177:1—177:37. URL: <https://www.uantwerpen.be/en/staff/jens-lemmens/publications/>.
 - [12] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever и Karl Cobbe. *Let's verify step by step*. OpenAI / arXiv preprint. 2023. URL: <https://arxiv.org/abs/2305.20050>.
 - [13] Thomas Loakman, William Thorne и Chien-Sheng Lin. «Comparing apples to oranges: A dataset & analysis of LLM humour understanding from traditional puns to topical jokes». B: *Findings of the Association for Computational Linguistics: EMNLP 2025*. 2025, с. 9502—9518. URL: <https://doi.org/10.18653/v1/2025.findings-emnlp.505>.
 - [14] Thomas Loakman, William Thorne и Chien-Sheng Lin. *Who's laughing now? An overview of computational humour generation and explanation*. arXiv:2509.21175. 2025. URL: <https://arxiv.org/abs/2509.21175>.
 - [15] Graeme Ritchie. «Current directions in computational humour». B: *Artificial Intelligence Review* 16.2 (2001), с. 119—135. URL: <https://doi.org/10.1023/A:1011610210506>.

- [16] Ryo Sakabe, Hwiyeon Kim, Tatsuya Hirasawa и Mamoru Komachi. *Assessing the capabilities of LLMs in humor: A multi-dimensional analysis of Oogiri generation and evaluation*. arXiv:2511.09133. 2025. URL: <https://arxiv.org/abs/2511.09133>.
- [17] Alexey Tikhonov и Pavel Shtykovskiy. *Humor mechanics: Advancing humor generation with multistep reasoning*. arXiv:2405.10073. 2024. URL: <https://arxiv.org/abs/2405.10073>.
- [18] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc Le и Denny Zhou. *Chain-of-thought prompting elicits reasoning in large language models*. arXiv:2201.11903. 2022. URL: <https://doi.org/10.48550/arXiv.2201.11903>.
- [19] Orion Weller и Kevin Seppi. «The rJokes dataset: A large scale humor collection». B: *Proceedings of the Twelfth Language Resources and Evaluation Conference*. 2020, с. 6136—6141. URL: <https://aclanthology.org/2020.lrec-1.753/>.
- [20] Tim Winters и Sam Van der Stockt. «Evaluating humor generation in an improvisational comedy setting». B: *Computational Linguistics in the Netherlands Journal* 14 (2025), с. 505—523. URL: <https://www.clinjournal.org/clinj/article/view/214>.
- [21] Zhen Xu, Shaojie Yuan, Li Chen и Diyi Yang. «“A good pun is its own reword”: Can large language models understand puns?» B: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, с. 11766—11782. URL: <https://doi.org/10.18653/v1/2024.emnlp-main.657>.
- [22] Tianyu Yu, Bin Ji, Shun Wang, Shunyu Yao, Zhijian Wang, Ganqu Cui, Luqi Yuan, Ning Ding, Yong Yao, Zhiyuan Liu, Maosong Sun и Tat-Seng Chua. *RLPR: Extrapolating RLVR to general domains without verifiers*. arXiv:2506.18254. 2025. URL: <https://doi.org/10.48550/arXiv.2506.18254>.
- [23] Jiawei Zhang, Sizhe Luo, Ruijie Zhang и Qi Su. *HUMORCHAIN: Theory-guided multi-stage reasoning for interpretable multimodal humor generation*. arXiv:2511.21732. 2025. URL: <https://doi.org/10.48550/arXiv.2511.21732>.

- [24] Jie Zhang, Lily Jain, Yichi Guo, Justin Chen, Kexin Linda Zhou, Sreeranjani Suresh, Amanda Wagenmaker, Scott Sievert, Taylor Rogers, Kevin Jamieson, Robert Mankoff и Robert Nowak. *Humor in AI: Massive scale crowd-sourced preferences and benchmarks for cartoon captioning*. Advances in Neural Information Processing Systems, Datasets and Benchmarks Track. 2024. URL: <https://arxiv.org/abs/2406.10522>.
- [25] Shaoxiong Zhong, Zicheng Huang, Shufan Gao, Wen Wen, Lin Lin, Marinka Zitnik и Pan Zhou. «Let’s think outside the box: Exploring leap-of-thought in large language models with creative humor generation». B: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, c. 13246—13257. URL: https://openaccess.thecvf.com/content/CVPR2024/html/Zhong_Lets_Think_Outside_the_Box_Exploring_Leap-of-Thought_in_Large_Language_CVPR_2024_paper.html.
- [26] Xingcheng Zhou, Zihan Liu, Ariel Sims, Han Wang, Tianyu Pang, Chao Li, Li Wang, Min Lin и Chao Du. *Reinforcing general reasoning without verifiers*. arXiv:2505.21493. 2025. URL: <https://arxiv.org/abs/2505.21493>.

Приложение А

Реализованные шаблоны запросов

В этом приложении фиксируются основные шаблоны, уже реализованные в репозитории. Они включены сюда, поскольку задают операциональную форму запросы, используемых на протяжении пилотного эксперимента.

А.1 Запрос для генерации эталонных ответов

Запрос, используемый для поиска эталонных ответов и генерацию кандидатов:

Напиши шутка используя следующих ключевое слово(s): {ключевые слова}

Его главное преимущество — согласованность. Одна и та же текстовая форма используется для запроса для векторного представления во время поиска и для последующей генерации кандидатов, что уменьшает несоответствие между построением набора данных и оценкой. Главный недостаток состоит в том, что шаблон намеренно общий и поэтому не кодирует ни теорию юмора, ни информацию об аудитории.

А.2 Инструкции для запросов для векторного поиска

Для адаптации моделью векторных представлений к поиску подзадачи используются два коротких инструкционных шаблона:

Instruct: Given ключевое слово для шутки, извлекать шутки
would совпадение ключевое слово.
Query: {ключевое слово}

Instruct: Given запрос для шутка, извлекать шутки что
would ответ запрос.
Query: {запрос}

Эти шаблоны важны, потому что пилотный не использует исходные строки ключевых слов как векторных представлений напрямую. Он кодирует их внутри инструкции, ориентированные на поиск, что делает векторное пространство семантического намерения более специализированным под задачу.

А.3 Оценочный запрос

Системная инструкция для попарная оценка просит оценщик сравнить две шутки, написанные для одного запроса, и выбрать ровно одного победитель. Инструкция явно не поощряет вознаграждать длину или стилистическую выверенность, если они не улучшают юмор. Такой дизайн намеренно прост и должен рассматриваться как базовая модель протокол оценки, а не как финальная рубрика, согласованная с человеческими оценками.

Приложение В

Дополнительные замечания о программной архитектуре

Репозиторий организован как набор асинхронный процессы:

- `JokesPipeline`: скачивает и предобрабатывает сырые корпуса;
- `EmbeddingsPipeline`: вычисляет плотные векторные представления шуток;
- `KeywordsPipeline`: извлекает значимые ключевые слова через сходство и MMR;
- `ReferencesPipeline`: строит наборы данных вида «запрос — эталонные ответы» через векторный поиск;
- `CandidatesPipeline`: генерирует шутки по запросам;
- `EvaluationPipeline`: выполняет попарное оценивание и агрегацию рейтинга.

Эта архитектура важна для финального диплома, потому что она отделяет построение данных от сравнения моделей. В результате последующий этап обучения MRVF можно добавить без переписывания остальной инфраструктуры. Отсутствующий компонент — именно тот, который теперь становится основной целью будущей работы: процесс обучения, который потребляет построенный запрос-набор эталонных ответов и оптимизирует формальную целевую функцию, введенную в Chapter 2.